

Node.js + MySQL CRUD API

Practical Database Connection, REST API Development & Postman Testing

Technology Stack: Node.js, Express.js, MySQL, phpMyAdmin, mysql2, Postman

Project: Student Management REST API

This tutorial starts from creating a MySQL database in phpMyAdmin and ends with testing complete Create, Read, Update and Delete APIs in Postman.

What you will build

- A Node.js + Express REST API
- A MySQL database managed through phpMyAdmin
- A reusable database connection using mysql2
- Student CRUD APIs
- Validation and error handling
- Postman requests for every CRUD operation

1. Tutorial Roadmap

- Prerequisites and software setup
- Create the MySQL database and table in phpMyAdmin
- Create and initialize the Node.js project
- Install Express and MySQL dependencies
- Create the database connection
- Create the Express server
- Create CRUD API routes
- Test every endpoint in Postman
- Troubleshooting common errors
- Recommended project structure and next steps

2. Prerequisites

Install the following before starting:

- Node.js — install the current LTS release. Verify with `node -v`.
- npm — normally installed with Node.js. Verify with `npm -v`.
- MySQL + phpMyAdmin — XAMPP, WAMP, Laragon, or another MySQL/phpMyAdmin installation is fine.
- Postman — used to send and inspect API requests.

Verify Node.js

```
node -v
npm -v
```

Start MySQL and phpMyAdmin

If you use XAMPP, start Apache and MySQL from the XAMPP Control Panel, then open phpMyAdmin in your browser.

Note: The exact phpMyAdmin URL can differ by your local installation. Common local installations use a localhost address.

3. Create Database in phpMyAdmin

Open phpMyAdmin and create a database named `student_api`. Then open the SQL tab and run the following SQL.

```
CREATE DATABASE student_api;

USE student_api;

CREATE TABLE students (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  course VARCHAR(100) NOT NULL,
  age INT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
  ON UPDATE CURRENT_TIMESTAMP
);
```

After execution, the database should contain a `students` table with fields for ID, name, email, course, age and timestamps.

Optional sample data

```
INSERT INTO students (name, email, course, age)
VALUES
('Rahul Sharma', 'rahul@example.com', 'MERN Stack', 22),
('Priya Patel', 'priya@example.com', 'Node.js', 21);
```

You can now use phpMyAdmin to confirm that the records exist.

4. Create the Node.js Project

```
mkdir student-api
cd student-api
npm init -y
```

Install dependencies

```
npm install express mysql2 cors dotenv
npm install --save-dev nodemon
```

Package purpose:

- express — HTTP server and API routing
- mysql2 — connects Node.js to MySQL
- cors — allows cross-origin requests when required by frontend applications
- dotenv — loads environment variables from .env
- nodemon — automatically restarts the server during development

Update package.json scripts

```
{
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  }
}
```

Recommended project structure

```
student-api/
├── config/
│   └── db.js
├── routes/
│   └── studentRoutes.js
├── .env
├── .gitignore
├── server.js
└── package.json
```

Note: Keep database credentials in .env and never commit .env to a public Git repository.

5. Configure Environment Variables

Create .env

```
PORT=5000

DB_HOST=localhost
DB_USER=root
DB_PASSWORD=
DB_NAME=student_api
DB_PORT=3306
```

If your MySQL root user has a password, put that password in DB_PASSWORD. Do not add quotes unless your password actually contains them.

Create .gitignore

```
node_modules/
.env
```

Create config/db.js

```
const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  port: Number(process.env.DB_PORT),
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});

module.exports = pool;
```

Note: A connection pool is preferred for an API because multiple requests can reuse database connections instead of opening a new connection for every request.

6. Create the Express Server

server.js

```
require('dotenv').config();

const express = require('express');
const cors = require('cors');
const studentRoutes = require('./routes/studentRoutes');

const app = express();

app.use(cors());
app.use(express.json());

app.get('/', (req, res) => {
  res.json({
    success: true,
    message: 'Student API is running'
  });
});

app.use('/api/students', studentRoutes);

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Start the server:

```
npm run dev
```

Open <http://localhost:5000/>. You should receive a JSON response confirming that the API is running.

7. Build the CRUD API

Create routes/studentRoutes.js

```
const express = require('express');
const router = express.Router();
const db = require('../config/db');

// CREATE - POST /api/students
router.post('/', async (req, res) => {
  try {
    const { name, email, course, age } = req.body;

    if (!name || !email || !course) {
      return res.status(400).json({
        success: false,
        message: 'name, email and course are required'
      });
    }

    const sql = `
      INSERT INTO students (name, email, course, age)
      VALUES (?, ?, ?, ?)
    `;

    const [result] = await db.execute(sql, [
      name,
      email,
      course,
      age || null
    ]);

    const [rows] = await db.execute(
```

```

        'SELECT * FROM students WHERE id = ?',
        [result.insertId]
    );

    res.status(201).json({
        success: true,
        message: 'Student created successfully',
        data: rows[0]
    });
} catch (error) {
    console.error(error);

    if (error.code === 'ER_DUP_ENTRY') {
        return res.status(409).json({
            success: false,
            message: 'Email already exists'
        });
    }

    res.status(500).json({
        success: false,
        message: 'Server error'
    });
}
});

// READ ALL - GET /api/students
router.get('/', async (req, res) => {
    try {
        const [rows] = await db.execute(
            'SELECT * FROM students ORDER BY id DESC'
        );

        res.json({
            success: true,
            count: rows.length,
            data: rows
        });
    } catch (error) {
        console.error(error);

        res.status(500).json({
            success: false,
            message: 'Server error'
        });
    }
});

// READ ONE - GET /api/students/:id
router.get('/:id', async (req, res) => {
    try {
        const { id } = req.params;

        const [rows] = await db.execute(
            'SELECT * FROM students WHERE id = ?',
            [id]
        );

        if (rows.length === 0) {
            return res.status(404).json({
                success: false,
                message: 'Student not found'
            });
        }

        res.json({
            success: true,
            data: rows[0]
        });
    } catch (error) {
        console.error(error);

        res.status(500).json({
            success: false,

```

```

        message: 'Server error'
    });
}
});

// UPDATE - PUT /api/students/:id
router.put('/:id', async (req, res) => {
  try {
    const { id } = req.params;
    const { name, email, course, age } = req.body;

    if (!name || !email || !course) {
      return res.status(400).json({
        success: false,
        message: 'name, email and course are required'
      });
    }

    const sql = `
      UPDATE students
      SET name = ?, email = ?, course = ?, age = ?
      WHERE id = ?
    `;

    const [result] = await db.execute(sql, [
      name,
      email,
      course,
      age || null,
      id
    ]);

    if (result.affectedRows === 0) {
      return res.status(404).json({
        success: false,
        message: 'Student not found'
      });
    }

    const [rows] = await db.execute(
      'SELECT * FROM students WHERE id = ?',
      [id]
    );

    res.json({
      success: true,
      message: 'Student updated successfully',
      data: rows[0]
    });
  } catch (error) {
    console.error(error);

    if (error.code === 'ER_DUP_ENTRY') {
      return res.status(409).json({
        success: false,
        message: 'Email already exists'
      });
    }

    res.status(500).json({
      success: false,
      message: 'Server error'
    });
  }
});

// DELETE - DELETE /api/students/:id
router.delete('/:id', async (req, res) => {
  try {
    const { id } = req.params;

    const [result] = await db.execute(
      'DELETE FROM students WHERE id = ?',
      [id]
    );
  }
});

```

```

    });

    if (result.affectedRows === 0) {
      return res.status(404).json({
        success: false,
        message: 'Student not found'
      });
    }

    res.json({
      success: true,
      message: 'Student deleted successfully'
    });
  } catch (error) {
    console.error(error);

    res.status(500).json({
      success: false,
      message: 'Server error'
    });
  }
});

module.exports = router;

```

Note: The ? placeholders are parameterized queries. They help prevent SQL injection and keep values separate from the SQL statement.

8. API Endpoint Summary

Method	Endpoint	Purpose
POST	/api/students	Create a student
GET	/api/students	Get all students
GET	/api/students/:id	Get one student
PUT	/api/students/:id	Update a student
DELETE	/api/students/:id	Delete a student

9. Test the API in Postman

Base URL: `http://localhost:5000`

A. CREATE — POST

Method: POST

URL: `http://localhost:5000/api/students`

In Postman select Body → raw → JSON and send:

```

{
  "name": "Amit Shah",
  "email": "amit@example.com",
  "course": "MERN Stack",
  "age": 23
}

```

Expected response: HTTP 201 Created with the inserted student.

```

{
  "success": true,
  "message": "Student created successfully",
  "data": {
    "id": 3,
    "name": "Amit Shah",
    "email": "amit@example.com",

```



```
    "course": "MERN Stack",
    "age": 23
  }
}
```

B. READ ALL — GET

Method: GET

URL: `http://localhost:5000/api/students`

No body is required. Expected response contains success, count and data.

```
{
  "success": true,
  "count": 3,
  "data": [
    {
      "id": 3,
      "name": "Amit Shah",
      "email": "amit@example.com",
      "course": "MERN Stack",
      "age": 23
    }
  ]
}
```

C. READ ONE — GET BY ID

Method: GET

URL: `http://localhost:5000/api/students/3`

Replace 3 with an actual student ID.

D. UPDATE — PUT

Method: PUT

URL: `http://localhost:5000/api/students/3`

```
{
  "name": "Amit Patel",
  "email": "amit@example.com",
  "course": "Node.js",
  "age": 24
}
```

Expected response: HTTP 200 OK and the updated record.

E. DELETE — DELETE

Method: DELETE

URL: `http://localhost:5000/api/students/3`

No body is required.

```
{
  "success": true,
  "message": "Student deleted successfully"
}
```

10. Complete Practical Testing Flow

- Start MySQL from XAMPP/WAMP/Laragon.
- Confirm the student_api database exists in phpMyAdmin.
- Start the Node.js API with `npm run dev`.
- Open Postman.
- Send POST to create a student.

- Send GET to confirm the student appears.
- Copy the returned ID.
- Send GET /api/students/:id to retrieve that exact student.
- Send PUT /api/students/:id to change the record.
- Send GET again to verify the update.
- Send DELETE /api/students/:id.
- Send GET again to confirm the record was removed.

Note: This sequence lets you verify the complete CRUD lifecycle instead of testing endpoints independently.

11. Understanding HTTP Status Codes

Code	Meaning	Used in this API
200	Request successful	GET, PUT, DELETE
201	Resource created	POST
400	Invalid/missing input	Validation failure
404	Resource not found	Invalid student ID
409	Conflict	Duplicate email
500	Server error	Unexpected backend/database error

12. Common Errors & Solutions

Error: ECONNREFUSED 127.0.0.1:3306

MySQL is probably not running, or MySQL is using a different port. Start MySQL and verify DB_PORT in .env.

Error: Access denied for user 'root'

Check DB_USER and DB_PASSWORD. If your local MySQL root account has a password, put it in DB_PASSWORD.

Error: Unknown database 'student_api'

Create the database in phpMyAdmin or run the CREATE DATABASE statement from this tutorial.

Error: ER_NO_SUCH_TABLE

Confirm that the students table exists inside the student_api database.

Error: ER_DUP_ENTRY

The email column is UNIQUE. Use a different email address or update the existing record.

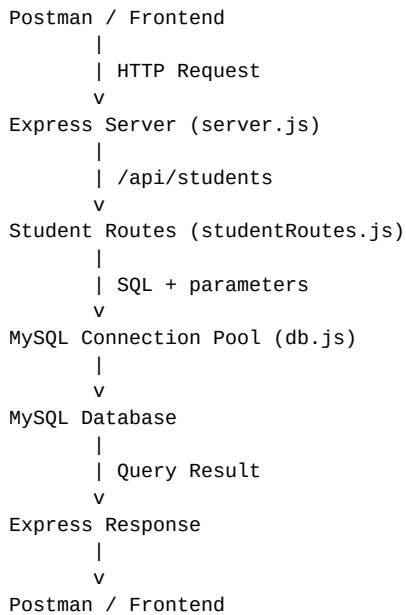
Postman returns Cannot GET /api/students

Confirm the route is mounted with app.use('/api/students', studentRoutes) and that the server was restarted.

req.body is undefined

Make sure app.use(express.json()) is placed before the routes and that Postman sends JSON with the Content-Type header application/json.

13. How the Request Travels Through the Application



This separation keeps the server entry point, database configuration and API routes easier to maintain.

14. Important Production Practices

- Never hard-code database passwords in source code.
- Never commit .env to Git.
- Use a dedicated MySQL user with only the permissions the application needs.
- Validate request data before running database operations.
- Use parameterized queries instead of concatenating user input into SQL.
- Add authentication and authorization before exposing sensitive CRUD endpoints.
- Use HTTPS in production.
- Add centralized error handling and request logging.
- Use migrations/schema management for larger projects.
- Back up the database and configure production monitoring.

15. Next-Level Improvements

- Add JWT authentication with login/register.
- Add role-based access such as admin and staff.
- Add pagination, search and sorting.
- Add Joi/Zod/express-validator validation.
- Separate controllers, services and routes as the application grows.
- Add Swagger/OpenAPI documentation.
- Connect a React/Next.js frontend to these APIs.
- Deploy the Node.js API and MySQL database to a production server.

16. Final Checklist

- ☐ MySQL is running
- ☐ student_api database exists
- ☐ students table exists
- ☐ .env contains correct DB credentials
- ☐ npm dependencies are installed
- ☐ server starts without errors
- ☐ GET / works
- ☐ POST creates a student
- ☐ GET returns students
- ☐ GET /:id returns one student
- ☐ PUT updates a student
- ☐ DELETE removes a student
- ☐ Errors return meaningful HTTP status codes

End of Practical Tutorial

You now have a complete Node.js + Express + MySQL CRUD API and a repeatable workflow for testing it through Postman.